URBAN RIPPLE

Predicting the Consequences of Temporary City Disruptions

Comprehensive Project Report

A web-based urban disruption intelligence and decision-support platform

Project	URBAN RIPPLE
Frontend	React.js with Vite
Backend	Python FastAPI with Uvicorn
Data layer evidenced in code	SQLite relational database
Report preparation date	2 November 2026
Repository reference	github.com/sumedhats/urban-ripple (upload status not independently confirmed)

Prepared from the project structure, source excerpts, API route inventory, dependency manifest and terminal verification outputs shared in this conversation. Features not independently evidenced are identified as planned, partial or requiring validation.

Contents

1. Executive Summary
2. Background and Problem Definition
3. Objectives and Scope
4. Proposed Solution and User Workflow
5. Technology Stack
6. System Architecture
7. Frontend Structure and User-Facing Areas
8. Backend Architecture
9. API Catalogue
10. Data Model and Persistence
11. Authentication, Roles and Security
12. Intelligence, Prediction and Scenario Capabilities
13. Analytics and Reporting
14. Installation and Execution
15. Verification and Test Evidence
16. Deployment and Repository Management
17. Limitations and Risks
18. Future Enhancements
19. Conclusion

Appendix A. Endpoint Reference

Appendix B. Evidence and Status Notes

Note: This is a technical project report based on evidence available at preparation time. It does not claim that every listed module is complete or production deployed.

1. Executive Summary

URBAN RIPPLE is a web-based urban disruption intelligence concept designed to help users understand how temporary disruptions can affect city services, infrastructure and residents. Examples of disruptions include road closures, flooding, utility interruptions and other time-bounded events. The central idea is to move beyond recording an incident and provide structured information for assessment, prioritisation and response planning.

The inspected project contains a React/Vite frontend and a Python FastAPI backend. The backend exposes API groups for authentication, disruptions, emergency incidents, intelligence outputs, impact analysis, analytics, machine-learning operations and scenario simulation. SQLite access is visible in backend code excerpts. Role-based access checks are implemented through authentication dependencies and role guards.

During the recorded verification session, the frontend production build completed successfully. Backend Python syntax compilation passed. FastAPI and Uvicorn dependencies were installed in a dedicated virtual environment, and the dependency consistency check reported no broken requirements. The running backend returned HTTP 200 at its root endpoint and at the API documentation endpoint. The browser-based frontend runtime and end-to-end workflows were not confirmed in the available verification record.

Project status at report time: backend API is running locally and frontend build succeeds; production deployment, end-to-end validation and GitHub push confirmation remain separate verification items.

2. Background and Problem Definition

Urban systems are interdependent. A temporary disruption in one location can affect movement, access to public services, utility availability and emergency response. Information about an incident may be spread across departments and systems, making it difficult to form a unified picture of the event and its wider consequences.

URBAN RIPPLE addresses this problem as a digital coordination and analysis platform. It provides a common workflow for registering disruptions, tracking incidents, retrieving risk and forecast information, examining potential impacts and simulating scenarios. The name reflects the project's focus on the chain of consequences that can spread from a local event into other parts of a city.

Problem statement

How can a city-oriented digital platform consolidate temporary disruption information and support the assessment of risk, potential impacts and response scenarios so that decision-makers can act with better context?

3. Objectives and Scope

- Provide a structured digital record of urban disruptions and emergency incidents.
- Expose APIs for creating, retrieving, updating and managing disruption records.
- Support authenticated access and role-based restrictions for sensitive operational functions.
- Present intelligence outputs such as risk scores, forecasts, anomalies, recommendations and multi-hazard information.
- Provide an impact-analysis operation and a scenario-simulation workflow.
- Expose summary and overview endpoints for dashboards and analytical views.
- Organise the application into frontend modules and backend service domains so that capabilities can be extended.

Scope boundary

The report describes functionality evidenced by source excerpts, module names and registered API routes. It does not assume live feeds from municipal sensors, official emergency systems, GIS data providers or external city databases unless such integrations are explicitly configured in the project. The model quality, forecast accuracy and operational benefit have not been independently benchmarked.

4. Proposed Solution and User Workflow

The proposed solution uses a browser-based interface connected to a REST-style FastAPI backend. The frontend calls shared API utilities, while the backend validates requests, applies authentication and role checks where configured, accesses stored records and returns JSON responses.

An indicative operational workflow is: (1) a user signs in; (2) a permitted user records a disruption or incident; (3) the record is stored and can be retrieved through management endpoints; (4) authorised users request risk, forecast, anomaly or recommendation outputs; (5) an analyst or official submits an impact-analysis or scenario-simulation request; and (6) overview and summary endpoints supply information for dashboard presentation.

Stage	Activity	Relevant API area
1. Access	Register, log in and retrieve current profile	/api/auth
2. Capture	Create and list disruption records	/api/disruptions
3. Manage	Retrieve, update or delete managed disruptions	/api/management/disruptions
4. Respond	Create and track emergency incidents	/api/emergency
5. Analyse	Request impact analysis, risk scores and forecasts	/api/impact and /api/intelligence
6. Plan	Inspect active disruptions and simulate scenarios	/api/scenarios
7. Monitor	Retrieve analytics overview and ML status	/api/analytics and /api/ml

5. Technology Stack

Layer	Technology / component	Role in the project
Frontend	React.js	Component-based user interface and client-side interaction.
Frontend tooling	Vite	Development server and production build tool.
Frontend API layer	JavaScript Fetch API; shared api.js utility	Centralises base URL selection and attaches bearer token when present.
Backend	Python, FastAPI	HTTP API, request handling, validation and routing.
ASGI server	Uvicorn	Runs the FastAPI application locally.
Persistence	SQLite (visible in backend code)	Relational storage accessed through sqlite3 connections.
Authentication	JWT-based token handling (visible in source excerpts)	Bearer-token authentication and current-user dependency.
Authorisation	Role dependency functions	Restricts selected routes to configured roles.
Data/ML libraries	NumPy, Pandas, SciPy, scikit-learn (listed in requirements)	Numerical processing and machine-learning support; exact model behaviour requires code-level validation.
Other backend packages	Pydantic, python-dotenv, httpx, joblib	Data validation, configuration, HTTP client and model utility support.

The backend requirements file also pins a number of transitive dependencies. The supplied requirements.txt should be treated as the authoritative installation manifest.

6. System Architecture

The high-level architecture is a browser client, a shared frontend API layer, a FastAPI application with domain routers, and a SQLite persistence layer. Some intelligence and ML routes form service boundaries within the backend. The exact runtime topology may differ in deployment.

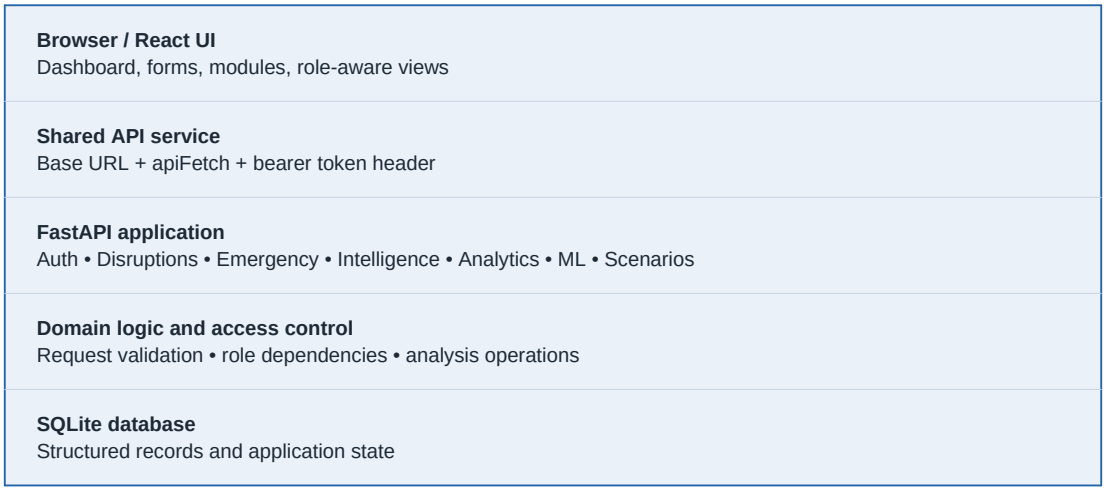


Figure 1. Logical request and data flow (conceptual, based on the inspected code structure).

7. Frontend Structure and User-Facing Areas

The frontend is organised under frontend/src and uses a central App.jsx entry composition together with AppRouter.jsx and a shared services layer. App.jsx imports API_BASE_URL and apiFetch from services/api.js. AppRouter.jsx and the authentication screen also reference the shared API base URL. Feature modules such as ScenarioSimulator and PredictiveIntelligence call the shared request utility.

The inspected source inventory includes role portal components, a citizen portal, role manager, resilience dashboard, system settings, system health, task management, incident timeline, water-risk area, public disruption tracking, predictive intelligence, scenario simulation, and service modules for analytics, authentication, emergency operations, ML, disruption records, reporting and language handling. The presence of a file is not by itself proof that the corresponding feature is fully implemented.

Frontend area / file evidence	Purpose indicated by its name and imports
App.jsx; AppRouter.jsx	Application shell and route composition.
auth/AuthScreen.jsx	Authentication screen.
portals/RolePortal.jsx; modules/RolePortals.jsx	Role-aware portal presentation.
modules/citizen/CitizenPortal.jsx	Citizen-facing portal area.
modules/permissions/RoleManager.jsx	Role/permission administration interface.
modules/ScenarioSimulator.jsx	Scenario simulation interface.
modules/PredictiveIntelligence.jsx	Predictive intelligence presentation.
modules/resilience/ResilienceDashboard.jsx	Resilience-oriented dashboard.
modules/timeline/IncidentTimeline.jsx	Incident timeline view.

Frontend area / file evidence	Purpose indicated by its name and imports
modules/system-health/SystemHealth.jsx	System health/status view.
services/api.js; authService.js; emergencyService.js; analyticsService.js; mlService.js; disruptionService.js	Client-side API/service abstraction.

The shared API helper selects VITE_API_BASE_URL, then VITE_API_URL, then a local fallback of http://127.0.0.1:8000. It reads the urban_ripple_token value from localStorage and adds an Authorization: Bearer header when a token is present. The frontend .env.example contains the local API base URL; the inspected environment-file listing showed only .env.example at the frontend root.

8. Backend Architecture

The backend entry point is backend/main.py. The recorded source inspection shows FastAPI application construction, CORS middleware and router registration. The active application registers routers for emergency incidents, intelligence, authentication, disruption management, analytics, ML and scenario operations. A wider backend directory inventory includes modules for alerts, citizen services, collaboration, departments, infrastructure, notifications, public API, resources, settings, tasks, timeline, users and workflows; not all of these were shown as mounted in the active entry point.

The code excerpts show SQLite connections using sqlite3 and row-based result handling. Authentication excerpts show JWT encoding/decoding, HTTP bearer credentials and a current-user dependency. Role utilities expose require_roles-style dependencies used by selected routers.

Backend domain	Responsibilities indicated by registered routes
Authentication	Registration, login and current-user profile.
Disruptions	Create, list and retrieve disruption records.
Disruption management	List and manage disruptions with update and delete operations.
Emergency	Create, list, retrieve, update incidents and retrieve summary.
Intelligence	Risk scores, forecast, anomalies, recommendations and multi-hazard output.
Impact	Accept an impact-analysis request.
Analytics	Provide an overview response.
ML	Expose model status, training trigger and anomaly retrieval.
Scenarios	List active-disruption scenarios and simulate a scenario.

9. API Catalogue

The following route inventory was returned by the running application's OpenAPI document during local verification. HTTP methods are shown exactly as observed.

Method	Endpoint	Purpose
GET	/	Backend identity/status response
GET	/health	Health endpoint
POST	/api/auth/register	Register account
POST	/api/auth/login	Authenticate and obtain token
GET	/api/auth/me	Retrieve current authenticated user

Method	Endpoint	Purpose
GET, POST	/api/disruptions	List and create disruption records
GET	/api/disruptions/{disruption_id}	Retrieve one disruption
GET	/api/management/disruptions	List managed disruptions
GET, PATCH, DELETE	/api/management/disruptions/{disruption_id}	Retrieve, update or delete a managed disruption
GET, POST	/api/emergency/incidents	List and create emergency incidents
GET	/api/emergency/incidents/{incident_id}	Retrieve one incident
PATCH	/api/emergency/incidents/{incident_id}	Update one incident
GET	/api/emergency/summary	Emergency incident summary
POST	/api/impact/analyze	Run impact analysis
GET	/api/intelligence/risk-scores	Retrieve risk scores
GET	/api/intelligence/forecast	Retrieve forecast output
GET	/api/intelligence/anomalies	Retrieve intelligence anomalies
GET	/api/intelligence/recommendations	Retrieve recommendations
GET	/api/intelligence/multi-hazard	Retrieve multi-hazard output
GET	/api/analytics/overview	Retrieve analytics overview
GET	/api/ml/status	Retrieve ML subsystem status
POST	/api/ml/train	Trigger ML training operation
GET	/api/ml/anomalies	Retrieve ML anomaly output
GET	/api/scenarios/active-disruptions	Retrieve active-disruption scenarios
POST	/api/scenarios/simulate	Submit a scenario simulation

This inventory contains 25 method/path entries. It should not be interpreted as 25 distinct business capabilities; some paths support multiple HTTP methods. Request schemas, response schemas, validation rules and error contracts should be taken from the generated OpenAPI specification.

10. Data Model and Persistence

SQLite usage is directly visible in backend excerpts through sqlite3.connect calls, row_factory configuration and database operations. The project folder also contained a local urban_ripple.db file during the developer's local audit. The exact complete schema, table relationships, indexes and migration strategy were not fully available in the evidence used to prepare this report.

At a conceptual level, the application needs to persist user accounts and role information, disruption records, emergency incident records, and any supporting model/scenario data required by the implemented services. This is a logical grouping inferred from the API surface; it is not a substitute for a verified entity-relationship diagram.

Data group	Fields / relationships to document from actual schema
Users	User identifier, login identity, password hash, role, account status and timestamps.
Disruptions	Disruption identifier, category/type, location, start/end or duration, severity, affected population and status where present.
Emergency incidents	Incident identifier, event details, severity, status, timestamps and responsible/related user where present.

Data group	Fields / relationships to document from actual schema
Analysis outputs	Input reference, risk/impact metrics, generated recommendations and analysis timestamp where persisted.
Scenarios	Scenario parameters, linked disruption(s), simulation output and creation metadata where persisted.

For a final academic submission, generate the ER diagram from the actual CREATE TABLE statements or schema file. Do not treat the conceptual groups above as confirmed table names.

11. Authentication, Roles and Security

Authentication code excerpts show an HTTP bearer scheme, a get_current_user dependency and JWT token creation and decoding. The frontend API helper attaches a bearer token from localStorage to requests. Registration, login and current-user routes are exposed under /api/auth.

Role-based access control is visible through security/roles.py and security/permissions/dependencies.py. The security audit excerpts show role guards on intelligence, emergency, management, analytics, ML and scenario routes. Roles explicitly referenced in route declarations include admin, official and analyst. The frontend inventory also includes role portal and role manager components.

- Keep secret keys, passwords, database files and local environment files out of source control.
- Use strong, environment-specific JWT signing secrets and rotate them if exposed.
- Store password hashes only; never store plaintext passwords.
- Validate role assignments on the server; do not rely on frontend visibility alone.
- Restrict CORS to trusted frontend origins in deployed environments.
- Use HTTPS in production and protect tokens against theft and cross-site scripting.
- Add audit logging for high-impact actions such as incident updates, deletion and ML training.

The code inventory indicates security controls, but it is not a formal security certification. Penetration testing, token lifecycle review, password policy review, rate limiting, backup encryption and deployment configuration still require separate verification.

12. Intelligence, Prediction and Scenario Capabilities

The intelligence router exposes risk scores, forecasts, anomaly detection, recommendations and multi-hazard outputs. The source excerpts show these endpoints protected for roles including admin, official and analyst. The project requirements list NumPy, Pandas, SciPy, scikit-learn and joblib, which are consistent with numerical analysis and ML workflows.

The ML router exposes status, training and anomaly endpoints. The scenario router exposes active-disruption retrieval and simulation. The impact-analysis endpoint accepts a POST request. Together, these APIs indicate a workflow that can combine incident information with analytical or predictive outputs to support planning.

Interpretation caution

The API names establish that these operations are exposed, not that a particular algorithm, training dataset, accuracy level or forecast horizon has been validated. The report therefore describes the functions at the service level and does not claim measured prediction performance.

13. Analytics and Reporting

The backend includes an analytics overview endpoint and an emergency summary endpoint. The frontend source inventory includes analyticsService.js, reportService.js and a resilience dashboard. These components indicate that the application has a reporting and dashboard-oriented structure.

Useful operational measures for this domain may include disruption counts by type and severity, incident status, affected population, duration, geographic distribution, response time and recurrence. These are suggested metrics for a mature dashboard; their presence in the live interface must be confirmed from the actual implementation.

14. Installation and Execution

The following commands reflect the local Mac development workflow used during verification. Run them from the project root. Node.js/npm must be installed for the frontend, and Python 3 must be installed for the backend.

Backend setup

```
cd ~/Desktop/urban-ripple
python3 -m venv backend/.venv
source backend/.venv/bin/activate
pip install -r backend/requirements.txt
cd backend
python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

The local API documentation is available at http://127.0.0.1:8000/docs while the backend is running.

Frontend setup

```
cd ~/Desktop/urban-ripple/frontend
npm install
npm run dev -- --host 127.0.0.1
```

The frontend development server commonly prints a local URL such as http://127.0.0.1:5173/. Use the exact URL printed by Vite. The frontend API base URL can be configured through VITE_API_BASE_URL; the shared API helper has a local fallback.

Production frontend build

```
cd ~/Desktop/urban-ripple/frontend
npm run build
```

15. Verification and Test Evidence

Check	Observed result	Interpretation
Frontend production build	Vite build completed; 2,868 modules transformed; output written to dist.	Build succeeded. A non-fatal warning reported a minified JavaScript chunk of about 1.50 MB.
Backend syntax compilation	python3 -m compileall -q backend completed successfully.	Python files passed syntax compilation; runtime behaviour is not guaranteed by this check.
Dependency installation	FastAPI 0.162.0 and Uvicorn 0.54.0 imported successfully in backend/.venv.	Required server packages are available in the dedicated virtual environment.
Dependency consistency	python -m pip check: No broken requirements found.	Installed package requirements are consistent.
Backend root endpoint	GET http://127.0.0.1:8000/ returned HTTP 200 and JSON status.	Backend was running locally at verification time.
API documentation	GET /docs returned HTTP 200.	FastAPI documentation page was served.
OpenAPI route inventory	25 method/path entries were listed from /openapi.json.	API router registration was confirmed.
Frontend browser runtime	Not confirmed in the final supplied output.	Must be tested manually in a browser.
End-to-end user workflows	Not confirmed by the supplied checks.	Requires authenticated UI-to-API tests.

Build warning: Vite reported a main minified JavaScript chunk of approximately 1,495.63 kB, above its 500 kB advisory threshold. Recommended remediation is route-level lazy loading and code splitting, then rebuilding to compare chunk sizes.

16. Deployment and Repository Management

The project has been prepared locally for Git version control. The root .gitignore excludes sensitive environment files, credential-like files, databases, Python virtual environments, node_modules, build outputs and backup patterns. The nested extra folder was verified as excluded from the root repository staging area.

The target repository referenced in the workflow is https://github.com/sumedhats/urban-ripple.git. The evidence available for this report showed that the local repository was on main with no commits yet at the time of a status check. A later successful git push output was not supplied. Therefore, this report does not claim that the GitHub repository contains the complete project.

Before publishing

- Review git status and git diff --cached --stat.
- Confirm that .env, credentials, private keys, local databases and virtual environments are not staged.
- Inspect empty non-init files and placeholders; the local audit reported 139 empty non-init files.
- Check that README.md contains accurate setup and feature documentation.
- Commit only reviewed project files.
- Inspect the remote branch before pushing; do not force-push over remote history.

17. Limitations and Risks

- Feature completeness is not established by file names or API route registration alone.

- The local audit found 139 empty non-init files, which may represent placeholders or unfinished modules and need review.
- The frontend production build succeeds, but its large JavaScript chunk may affect initial load performance.
- Backend syntax and dependency checks passed, but comprehensive automated tests and end-to-end flows were not demonstrated.
- The exact database schema and data dictionary were not fully inspected for this report.
- Prediction quality, training reproducibility, data provenance and model drift controls have not been benchmarked.
- Production hosting, HTTPS, environment-specific secrets, backups, monitoring and disaster recovery have not been confirmed.
- GitHub upload completion was not independently verified.

18. Future Enhancements

- Complete and test every placeholder module; remove unused or duplicate components after review.
- Introduce route-level lazy loading and code splitting to reduce the initial frontend bundle.
- Document the SQLite schema and add versioned migrations and reproducible seed data.
- Build automated API tests for authentication, role restrictions, CRUD operations, validation and error handling.
- Add end-to-end browser tests for registration, login, disruption creation, incident tracking, analytics and scenario simulation.
- Add geospatial visualisation with verified map layers, location validation and geographic filtering where the data supports it.
- Connect trustworthy real-time data sources only through documented, permissioned integrations.
- Evaluate forecasting and anomaly models using held-out data, baseline comparisons and appropriate metrics.
- Add audit trails, structured logging, rate limiting, monitoring, backup/restore tests and security reviews.
- Prepare a deployment configuration with environment-specific settings, HTTPS, managed secrets and operational documentation.

19. Conclusion

URBAN RIPPLE establishes a full-stack foundation for a city disruption intelligence platform. The verified API surface spans authentication, disruption management, emergency incidents, intelligence, impact analysis, analytics, ML operations and scenario simulation. The frontend is structured around React components and shared API services, while the backend uses FastAPI, role dependencies and SQLite access.

The local build and backend checks provide useful implementation evidence, but they are not equivalent to production readiness. The next engineering priorities are to verify all user-facing workflows, complete empty or placeholder modules, document the database schema, add automated tests, optimise frontend bundle size and confirm a secure deployment and successful repository publication.

Appendix A. Endpoint Reference

HTTP method	Path
GET	/
GET	/health
POST	/api/auth/register
POST	/api/auth/login
GET	/api/auth/me
GET, POST	/api/disruptions
GET	/api/disruptions/{disruption_id}
GET	/api/management/disruptions
GET, PATCH, DELETE	/api/management/disruptions/{disruption_id}
GET, POST	/api/emergency/incidents
GET	/api/emergency/incidents/{incident_id}
PATCH	/api/emergency/incidents/{incident_id}
GET	/api/emergency/summary
POST	/api/impact/analyze
GET	/api/intelligence/risk-scores
GET	/api/intelligence/forecast
GET	/api/intelligence/anomalies
GET	/api/intelligence/recommendations
GET	/api/intelligence/multi-hazard
GET	/api/analytics/overview
GET	/api/ml/status
POST	/api/ml/train
GET	/api/ml/anomalies
GET	/api/scenarios/active-disruptions
POST	/api/scenarios/simulate

Appendix B. Evidence and Status Notes

- Frontend build output: Vite 8.3.5; 2,868 modules transformed; build completed in approximately 573 ms; one large-chunk warning.
- Backend syntax check: python3 -m compileall -q backend returned success.
- Dependency check: FastAPI 0.162.0, Uvicorn 0.54.0; pip check reported no broken requirements.
- Backend local checks: root endpoint and /docs returned HTTP 200; /openapi.json returned the route inventory reproduced in this report.

- Source inspection: frontend/src/App.jsx, AppRouter.jsx, services/api.js, auth/AuthScreen.jsx and selected feature modules; backend/main.py, authentication, role, disruption, emergency, intelligence, analytics, ML and scenario route excerpts.
- Repository check: local branch main was observed with no commits at one point in the supplied Git workflow output. Remote push success was not evidenced.
- Sensitive credentials and account identifiers appearing in terminal-history uploads were intentionally excluded from this report.

End of Report